DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS****
PROJECT NAME	HEMATOVISION: ADVANCED BLOOD CELL CLASSIFICATION USING TRANSFER LEARNING
MAXIMUM MARKS	4 MARKS

6.5 APPLICATION BUILDING :

Application building is the final stage of the HematoVision project, where the trained deep learning model is integrated into a functional web-based system. The objective is to create a user-friendly interface that allows users to upload microscopic blood cell images and receive instant classification results.

The front-end of the application is developed using HTML and CSS to design an interactive and visually clear user interface. The UI includes options to upload an image, submit it for prediction, and display the predicted blood cell type along with a confidence score. The layout is designed to be simple and accessible for laboratory technicians and healthcare professionals.

The back-end is developed using a Python-based framework such as Flask. The server-side script handles image upload, performs preprocessing (resizing, normalization, and formatting), and loads the saved trained model ("blood_cell.h5"). The processed image is then passed to the model for prediction.

After prediction, the result is sent back to the front-end and displayed on the webpage. The application ensures smooth communication between the user interface and the deep learning model. Error handling mechanisms are also implemented to manage invalid inputs or upload issues.

Overall, the application building phase transforms the trained MobileNetV2 model into a practical, real-world solution, enabling automated blood cell classification through an easy-to-use web platform.

```
from flask import Flask, render_template, request
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import os
import base64
from io import BytesIO
from PIL import Image

app = Flask(__name__)
```

```
model = tf.keras.models.load_model("blood_cell_model.h5")

UPLOAD_FOLDER = "static/uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# ⚠ MUST MATCH TRAINING OUTPUT
class_labels = ['eosinophil', 'lymphocyte', 'monocyte', 'neutrophil']

@app.route('/', methods=['GET', 'POST'])
def home():
    if request.method == 'POST':
        file = request.files['file']
        filepath = os.path.join(UPLOAD_FOLDER, file.filename)
        file.save(filepath)
        img = load_img(filepath, target_size=(224,224))
        img_array = img_to_array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        prediction = model.predict(img_array)
        predicted_class = class_labels[np.argmax(prediction)]
        # Convert image to base64 for result page
        img = Image.open(filepath)
        buffered = BytesIO()
        img.save(buffered, format="PNG")
        img_data = base64.b64encode(buffered.getvalue()).decode()

        return render_template(
            'result.html',
            class_label=predicted_class,
            img_data=img_data
        )
    return render_template('home.html')

if __name__ == "__main__":
    app.run(debug=True)
```